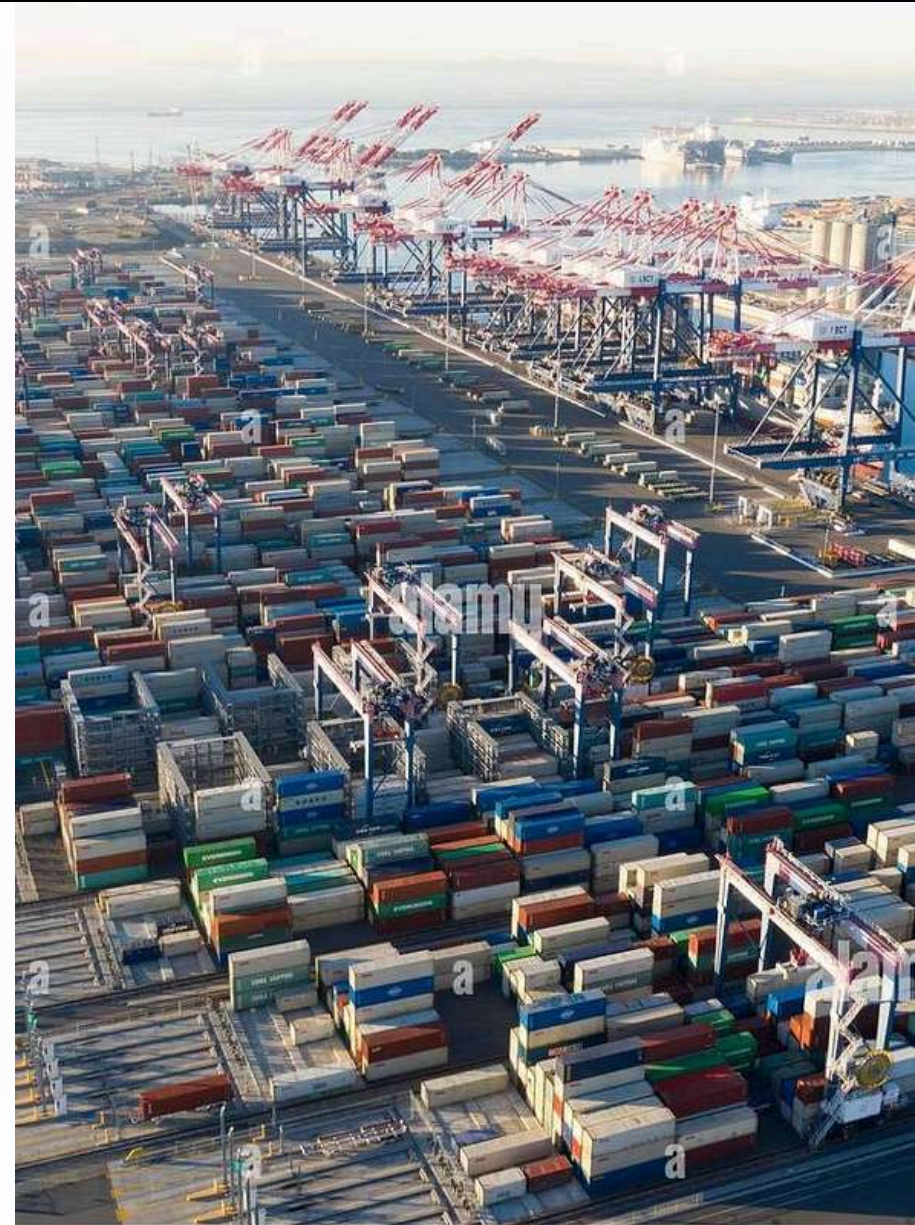


Day 3: Docker & Introduction to Jenkins

CI/CD with Jenkins and GenAI — Building the Foundation for Automated Pipelines

DAY 3 OF 5



Today's Agenda

01

Docker Basics

What Docker is and the problems it solves

02

Images vs. Containers

Architecture, flow, and key concepts

03

Docker Commands

Pull, run, stop, inspect, and port mapping

04

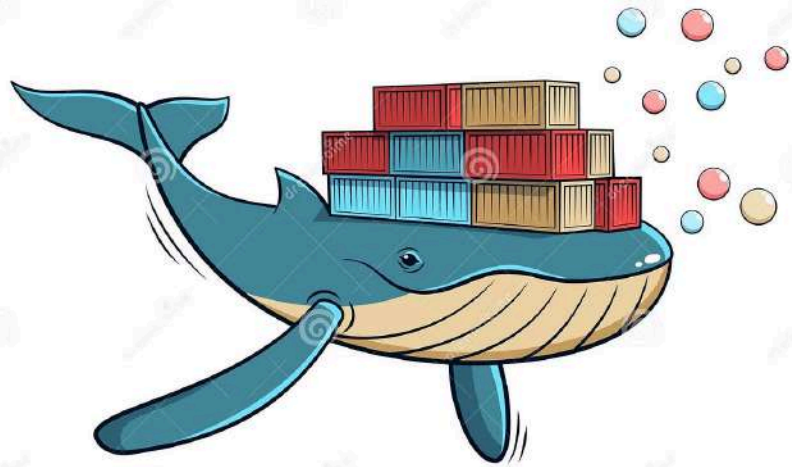
Dockerfile Creation

Writing, building, and running your own image

05

Jenkins Setup

Installation on EC2, unlock, and first login



[dreamstime.com](https://www.dreamstime.com/)

ID 308220170 © Agor2012

CHAPTER 1

Docker Basics

Understanding the problem Docker solves — and why it changed everything

What is Docker?

Docker is an open platform that lets you **package, ship, and run applications** inside lightweight, isolated environments called **containers**.



Package

Bundle code + dependencies
together



Ship

Move anywhere — laptop,
server, cloud



Run

Execute consistently in any environment



The "Works on My Machine" Problem



Sound familiar?

Code runs perfectly on your laptop — then breaks on the server. Different OS versions, missing dependencies, or conflicting libraries create unpredictable failures.

⚠ Without Docker: Every environment is a mystery.

✅ With Docker: One container, consistent behavior everywhere.

The Shipping Container Analogy



Before containers (shipping)

Every cargo item was loaded differently — fragile, slow, inconsistent.

After containers (shipping)

Standardized boxes stack anywhere — ship, truck, or rail. The same logic applies to Docker containers for software.

□ Your app is the cargo. Docker is the container. The host is the ship.

Why Docker? Key Benefits



Portability

Run the same container on any OS, cloud provider, or CI/CD system



Speed

Containers start in seconds vs. minutes for VMs



Consistency

Dev, staging, and production behave identically

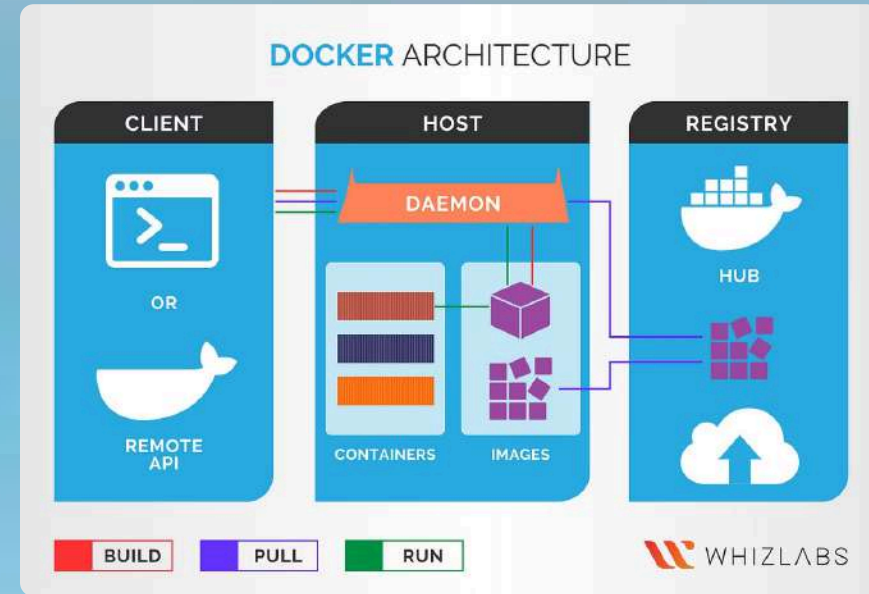


Isolation

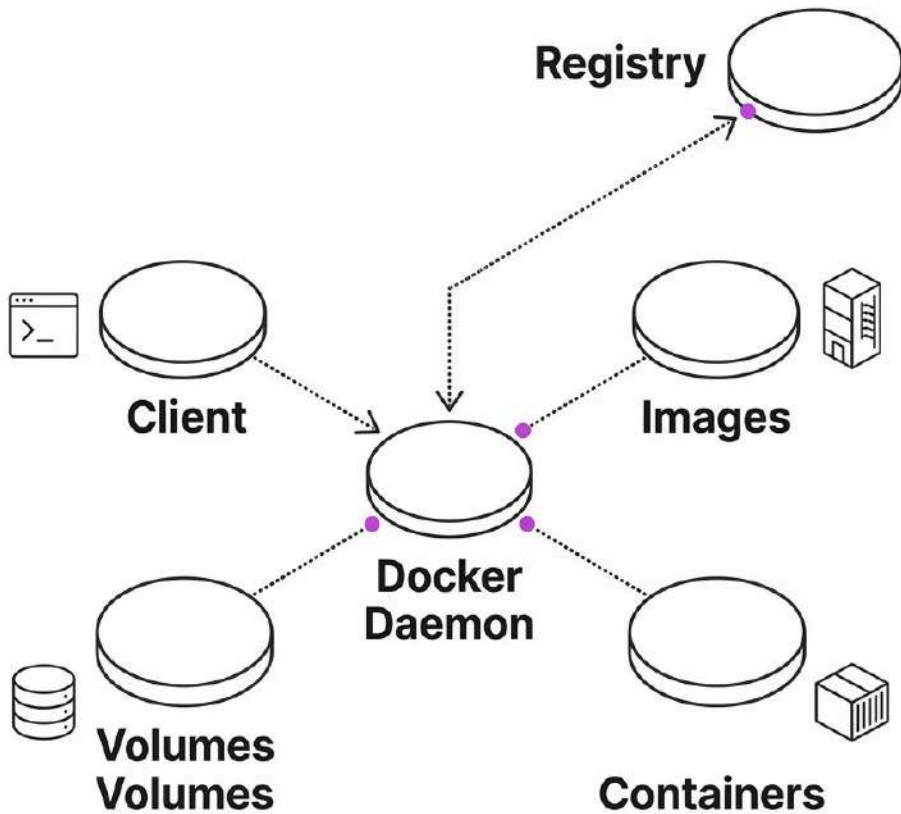
Each app runs in its own sandbox — no conflicts

Docker Architecture

How Docker Engine works — and the relationship between images and containers



Docker Engine Overview



Three core components

- **Docker Client** — CLI you type commands into
- **Docker Daemon** — Background service that does the work
- **Docker Registry** — Remote store for images (e.g., Docker Hub)



When you run `docker run`, the client talks to the daemon, which pulls the image from the registry and starts the container.

Images vs. Containers



Image

A **read-only template** — like a blueprint or a recipe. Defines what the app looks like.

- Stored in a registry
- Versioned with tags
- Reusable and shareable



Container

A **running instance** of an image — like a house built from a blueprint.

- Isolated process on the host
- Has its own filesystem
- Can be started, stopped, deleted



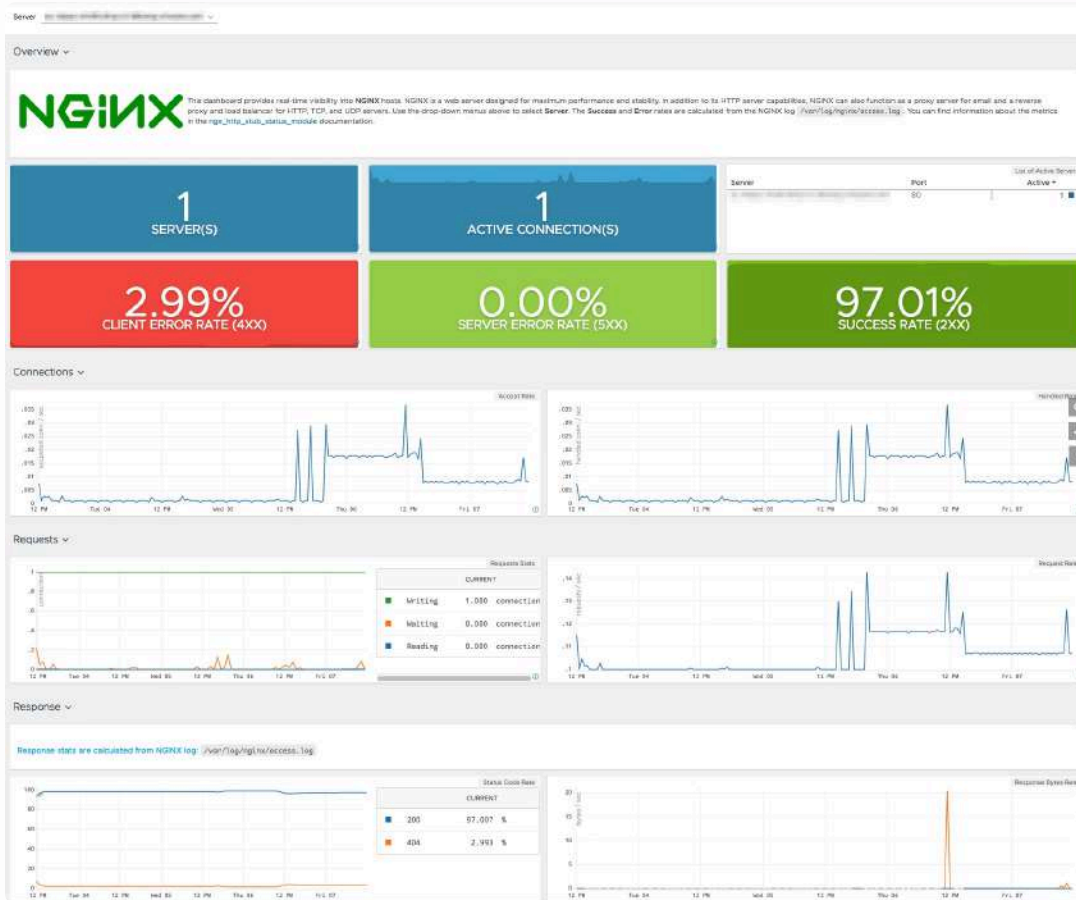
Same image → multiple containers. One recipe → many meals.

Image → Container Flow



Every container starts life as an image. You pull it once, then spin up as many containers as you need — instantly and identically.

Real-World Example: Web Server



Use case: NGINX in a container

Instead of installing NGINX on your host OS, you pull the official NGINX image and run it in a container. Swap versions in seconds with zero system pollution.

```
docker pull nginx  
docker run -d -p 80:80 nginx
```

✔ Your host stays clean. Your app stays portable.

CHAPTER 3

Docker Commands

The essential CLI toolkit every Docker practitioner needs to know

Essential Docker Commands

```
docker pull <image>
```

Download an image from Docker Hub to your local machine

```
docker run <image>
```

Create and start a container from an image

```
docker ps
```

List all **running** containers. Add `-a` to see stopped ones too

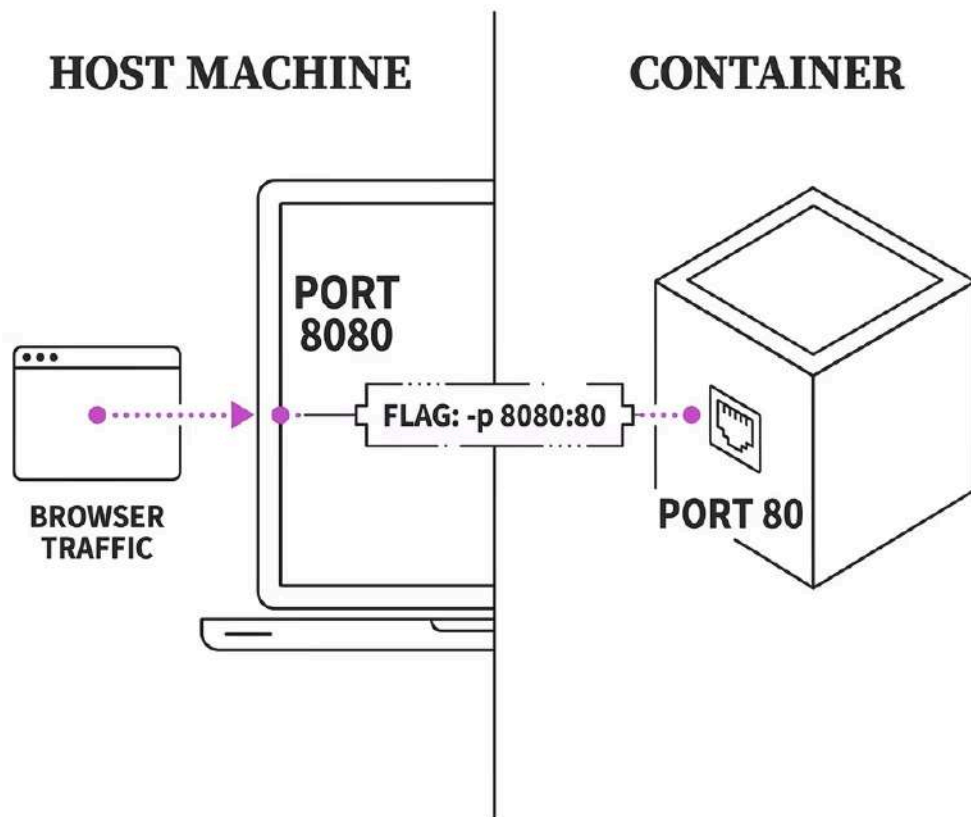
```
docker stop <id>
```

Gracefully stop a running container

```
docker rm <id>
```

Remove a stopped container permanently

Port Mapping: The -p Flag



Syntax: `-p HOST:CONTAINER`

Containers are isolated by default — ports must be explicitly mapped to be accessible from your machine or browser.

```
docker run -d -p 8080:80 nginx
```

- **8080** → port on your host machine
- **80** → port inside the container

 Visit `http://localhost:8080` to see your container running.

Running NGINX: Full Command Breakdown

1

```
docker pull nginx
```

Download the latest official NGINX image from Docker Hub

2

```
docker run -d -p 8080:80 nginx
```

-d runs in detached (background) mode; -p maps ports

3

```
docker ps
```

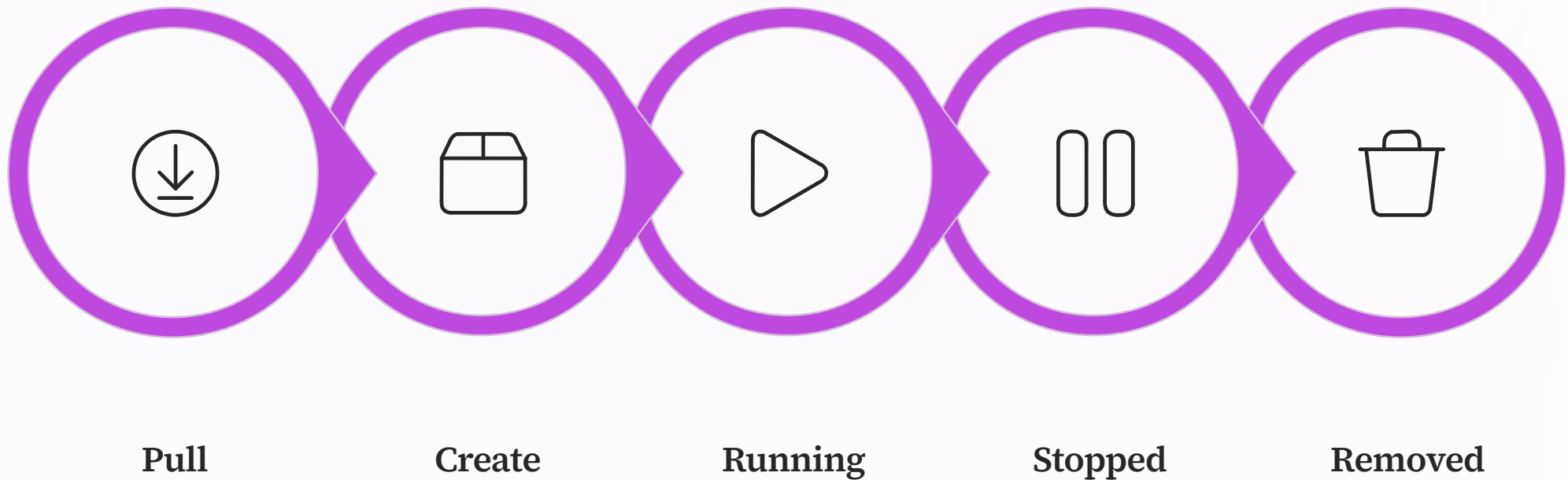
Confirm the container is running — note the container ID

4

Open browser → <http://localhost:8080>

You should see the NGINX welcome page

Container Lifecycle



Understanding the full lifecycle helps you manage resources efficiently — stopped containers still consume disk space until removed.

CHAPTER 4

Hands-On Lab

Run your first container and access it live in the browser



Lab: Running Your First Container

1 Pull the image

`docker pull nginx` — fetches the latest NGINX from Docker Hub

3 Verify it's running

`docker ps` — look for `my-nginx` in the list

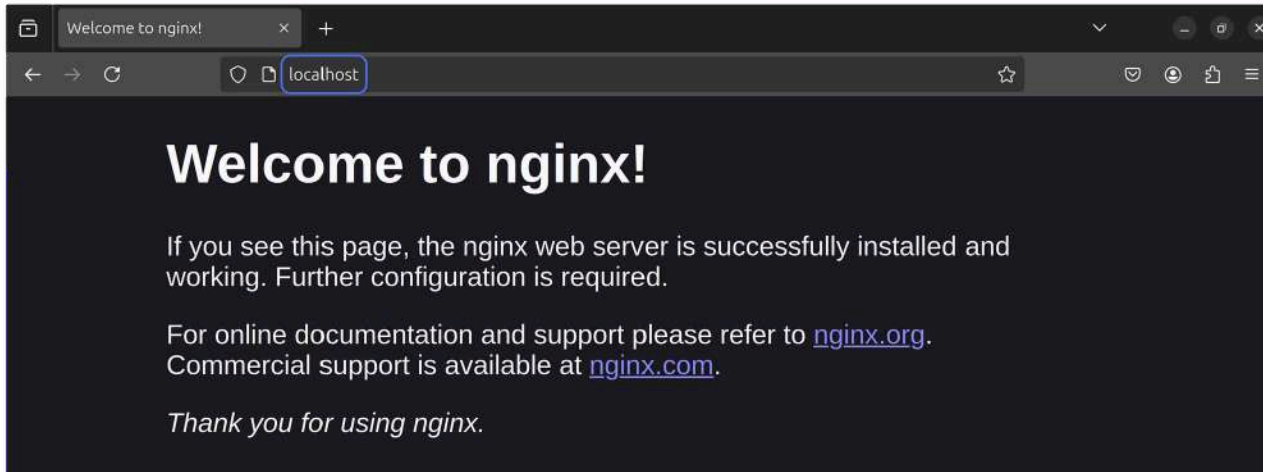
2 Start the container

`docker run -d -p 8080:80 --name my-nginx nginx`

4 Open your browser

Navigate to `http://localhost:8080` — NGINX welcome page should appear

Expected Output



✓ Success looks like this

The NGINX "Welcome to nginx!" page confirms your container is live and port mapping is working correctly.

What to check in terminal

```
CONTAINER ID  IMAGE  STATUS
a3f1b2c4d5e6  nginx  Up 2
minutes
```

✓ Container ID + "Up X minutes" = you're good to go!

Common Errors & Fixes

✗ Port already in use

Error: Bind: address already in use

Fix: Change host port — try -p 9090:80 instead

✗ Permission denied

Error: Got permission denied while trying to connect

Fix: Run `sudo usermod -aG docker $USER`, then log out and back in

✗ Container not found

Error: Error: No such container

Fix: Use `docker ps -a` to find the correct ID or name

CHAPTER 5

Dockerfile

Writing your own image blueprint — from scratch to production-ready



What is a Dockerfile?



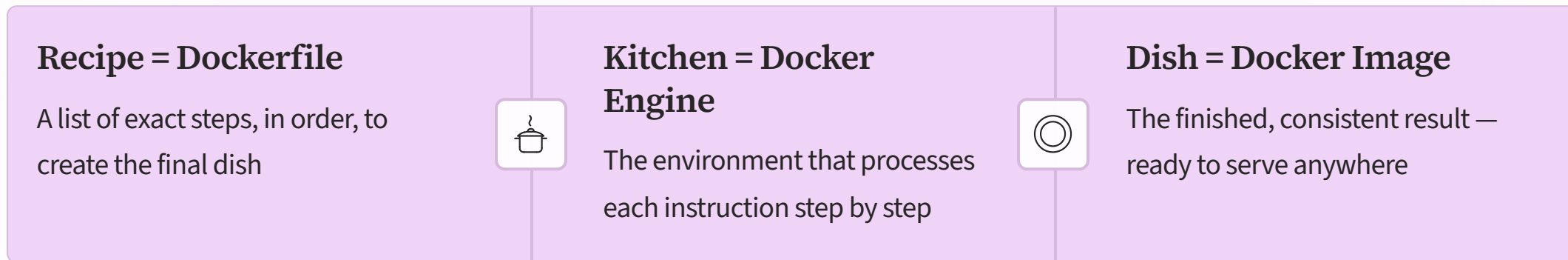
A text file that builds your image

A **Dockerfile** is a script of instructions that Docker follows to build a custom image. Think of it as a recipe — every line adds a new layer to your image.

- Reproducible builds every time
- Version-controlled with your code
- Shared across teams and environments

❏ No Dockerfile = no custom image. It's the starting point for everything.

The Recipe Analogy



Every time you run the same recipe in the same kitchen, you get the same dish. Every time you build the same Dockerfile, you get the same image.

Dockerfile: Core Instructions

FROM

Base image to build on top of — e.g., `FROM node:18`

WORKDIR

Set the working directory inside the container — e.g., `WORKDIR /app`

COPY

Copy files from host into the image — e.g., `COPY . .`

RUN

Execute a command during build — e.g., `RUN npm install`

CMD

Default command when container starts — e.g., `CMD ["node", "app.js"]`

CHAPTER 6

Dockerfile Examples

Real Node.js and Python examples — step by step

Sample Dockerfile: Node.js App

```
# Use official Node.js base image
FROM node:18-alpine

# Set working directory
WORKDIR /app

# Copy package files first (cache layer)
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy rest of the app
COPY . .

# Expose port
EXPOSE 3000

# Start the app
CMD ["node", "server.js"]
```

Why this order matters

Docker builds in **layers**. Copying `package.json` before source files means dependencies are only re-installed when they change — not on every code edit.



Alpine images are minimal Linux variants — much smaller than full Ubuntu-based images.

Sample Dockerfile: Python App

```
# Lightweight Python base
FROM python:3.11-slim

WORKDIR /app

# Install dependencies first
COPY requirements.txt .
RUN pip install -r requirements.txt

# Copy application code
COPY . .

EXPOSE 5000

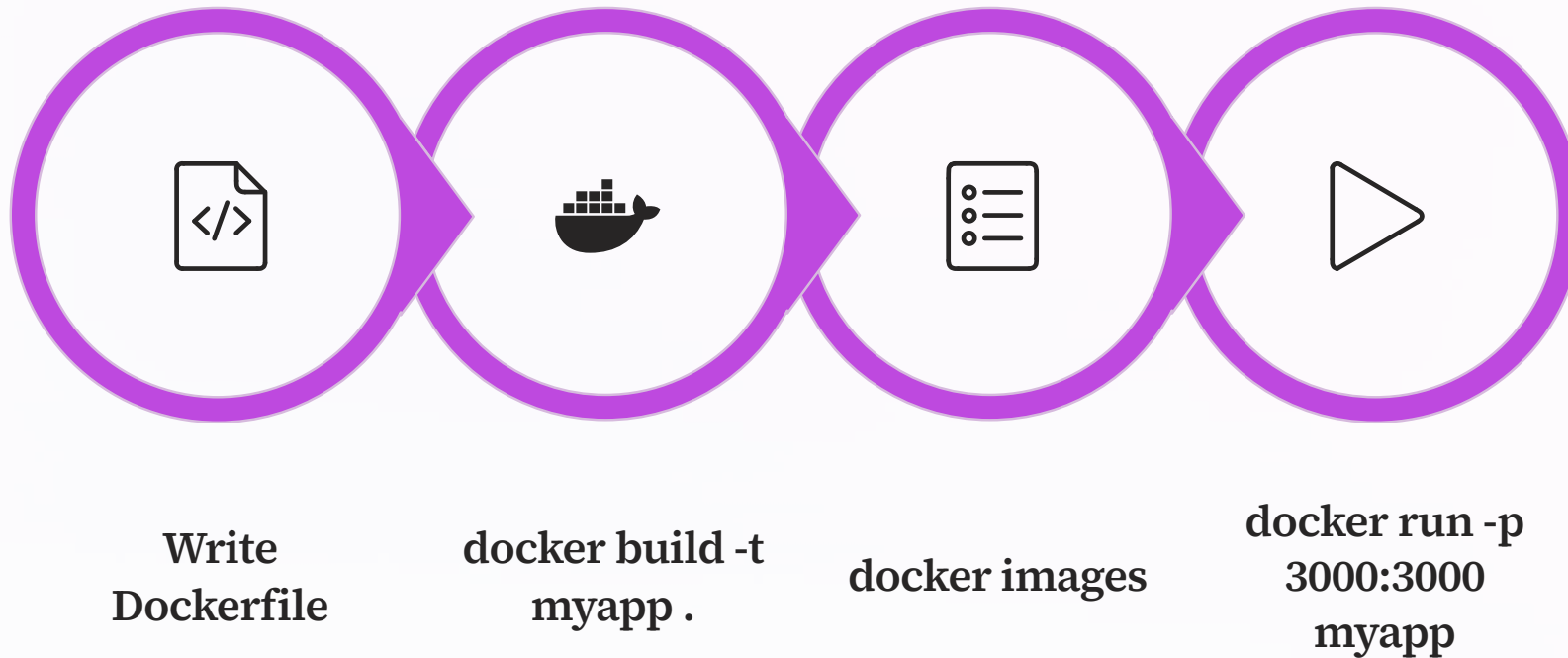
CMD ["python", "app.py"]
```

Same pattern, different stack

The structure mirrors the Node.js example. This consistency is intentional — it makes Dockerfiles predictable regardless of the language.

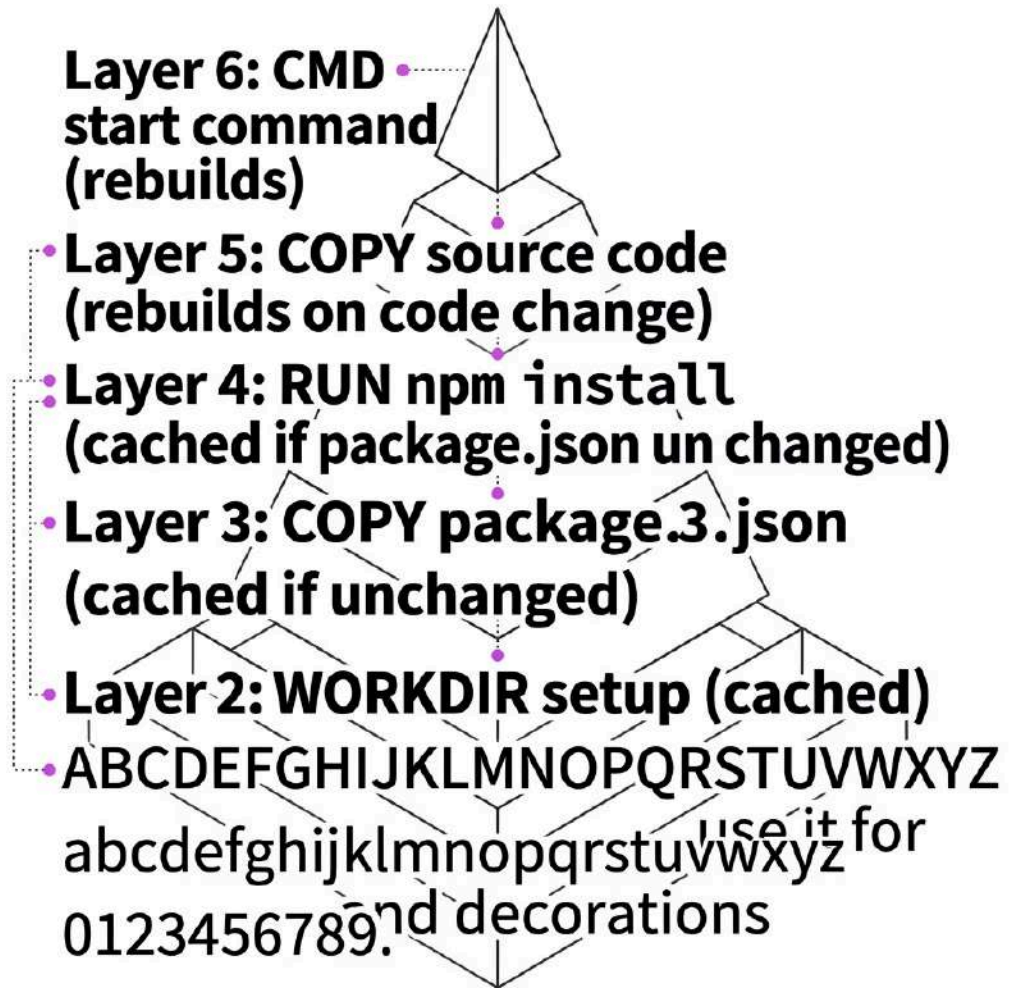
✓ Use `slim` or `alpine` variants to keep image size small and attack surface minimal.

Build & Run Flow



The `-t` flag tags your image with a name and optional version (e.g., `myapp:v1`). Always tag images — it makes managing them much easier.

Dockerfile Layer Caching

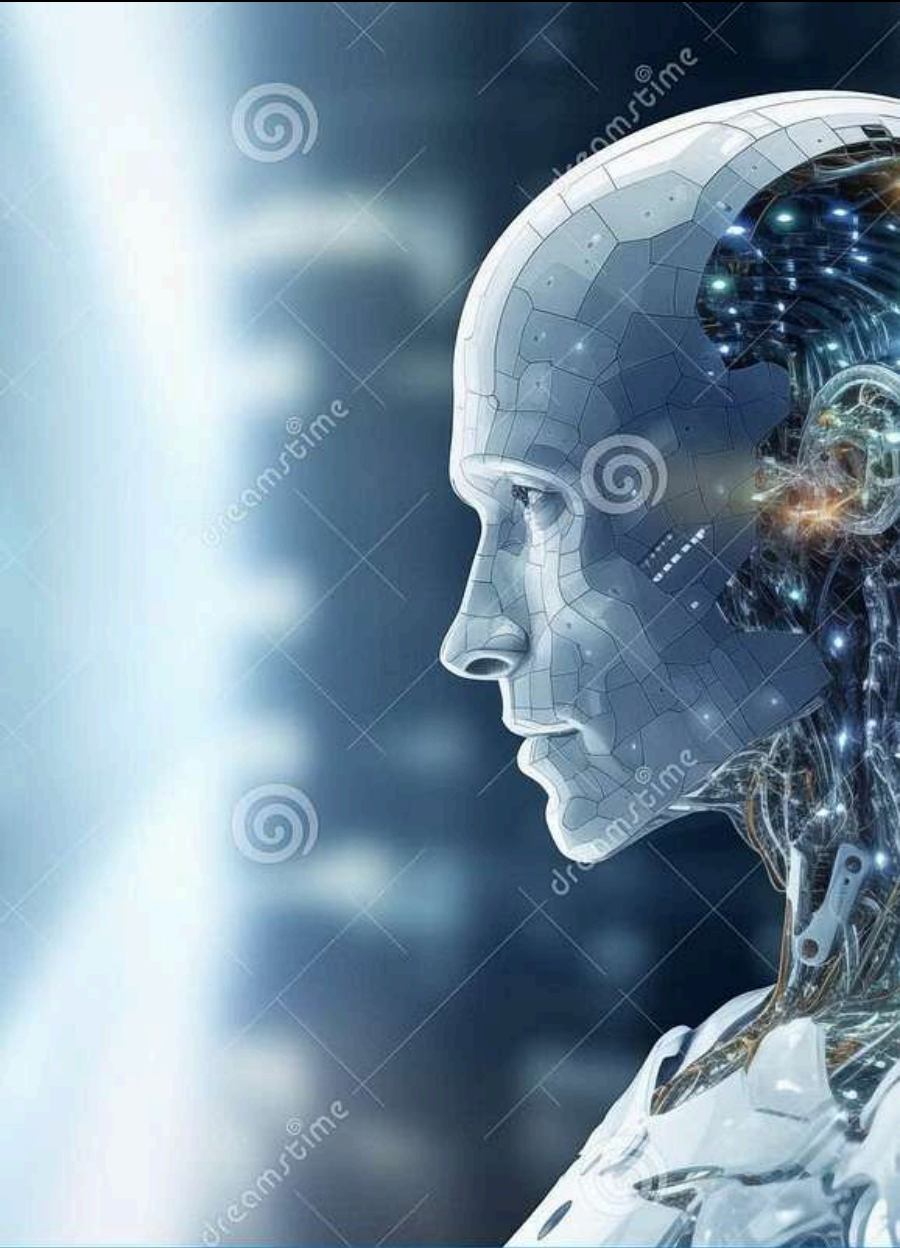


Faster builds through smart ordering

Docker caches each layer. If a layer hasn't changed, it reuses the cache. Place **slow, stable steps first** (dependencies) and **fast, changing steps last** (source code).



Good layer ordering can cut rebuild times from minutes to seconds.



CHAPTER 7

AI + Docker

Using ChatGPT and GitHub Copilot to write smarter, faster Dockerfiles

Generate a Dockerfile with ChatGPT



Prompt it like a senior engineer

Use specific, context-rich prompts to get production-ready output:

"Write a Dockerfile for a Python Flask app using Python 3.11. It must be lightweight, expose port 5000, and install dependencies from requirements.txt."

- ✓ Always review AI output — validate base image versions, exposed ports, and security best practices before using in production.

GitHub Copilot in VS Code

1 Open your project in VS Code

Create a new file named `Dockerfile` — Copilot recognizes the filename automatically

2 Type a comment to guide Copilot

`# Dockerfile for Node.js 18`
`Express app on port 3000 —`
Copilot fills in the rest

3 Accept, reject, or refine

Use `Tab` to accept suggestions, `Esc` to reject, or rephrase your comment for different results

Optimize Dockerfiles with AI Prompts

Reduce image size

"Optimize this Dockerfile to minimize the final image size using multi-stage builds."

Security hardening

"Rewrite this Dockerfile to run as a non-root user and remove unnecessary packages."

Build speed

"Reorder these Dockerfile instructions to maximize layer caching efficiency."

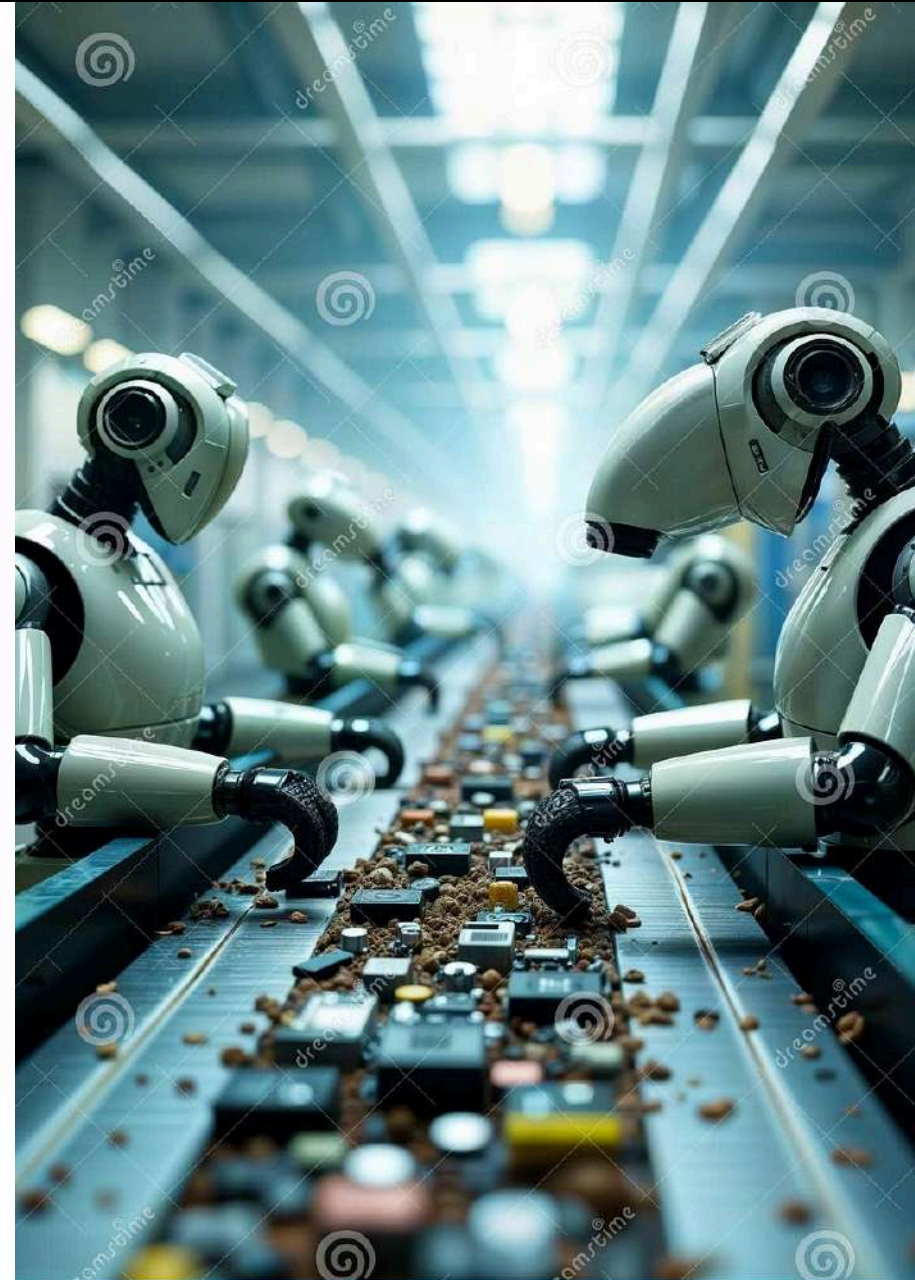


Treat AI as a pair programmer — it drafts, you review. Always verify before committing.

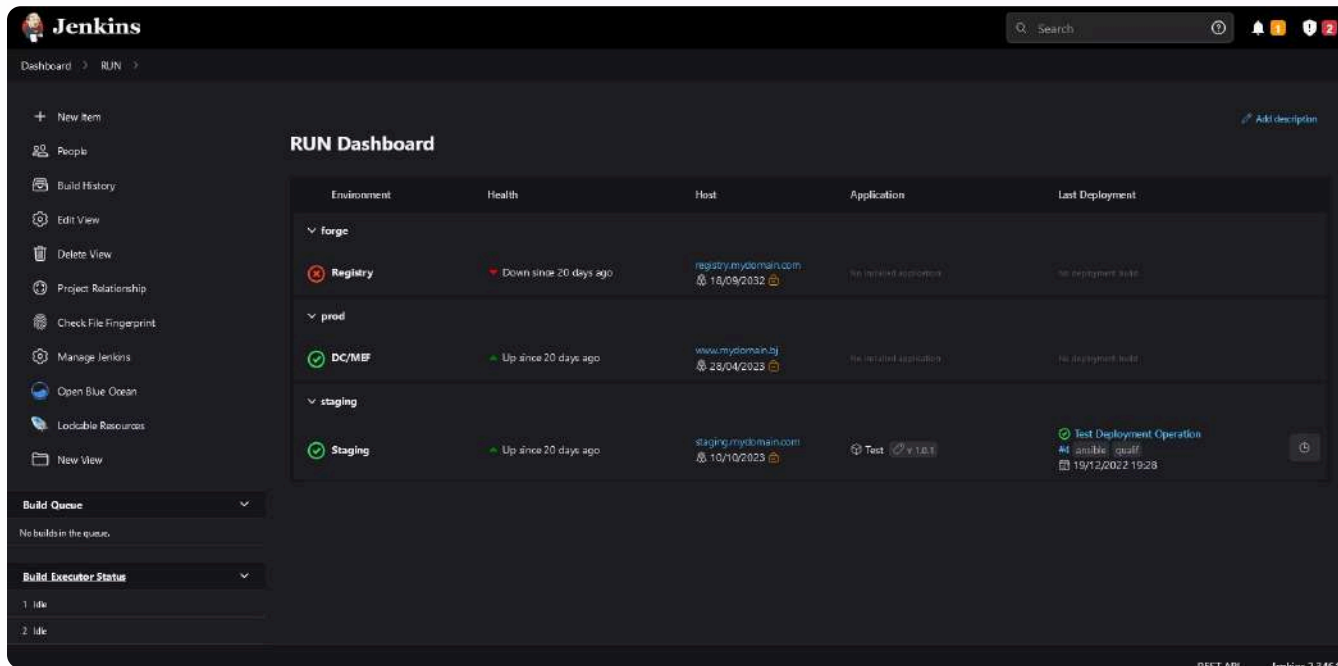
CHAPTER 8

Jenkins Introduction

Automating the path from code commit to deployed container



What is Jenkins?



The most widely used CI/CD automation server

Jenkins watches your source code, automatically triggers builds when changes are pushed, runs tests, builds Docker images, and deploys — all without human intervention.

- Open-source with 1,800+ plugins
- Runs on any OS or in Docker itself
- Integrates with GitHub, Docker, AWS, Slack, and more

Why Jenkins in CI/CD?



Automate Everything

No more manual builds or deployments — Jenkins reacts to every push



Catch Issues Early

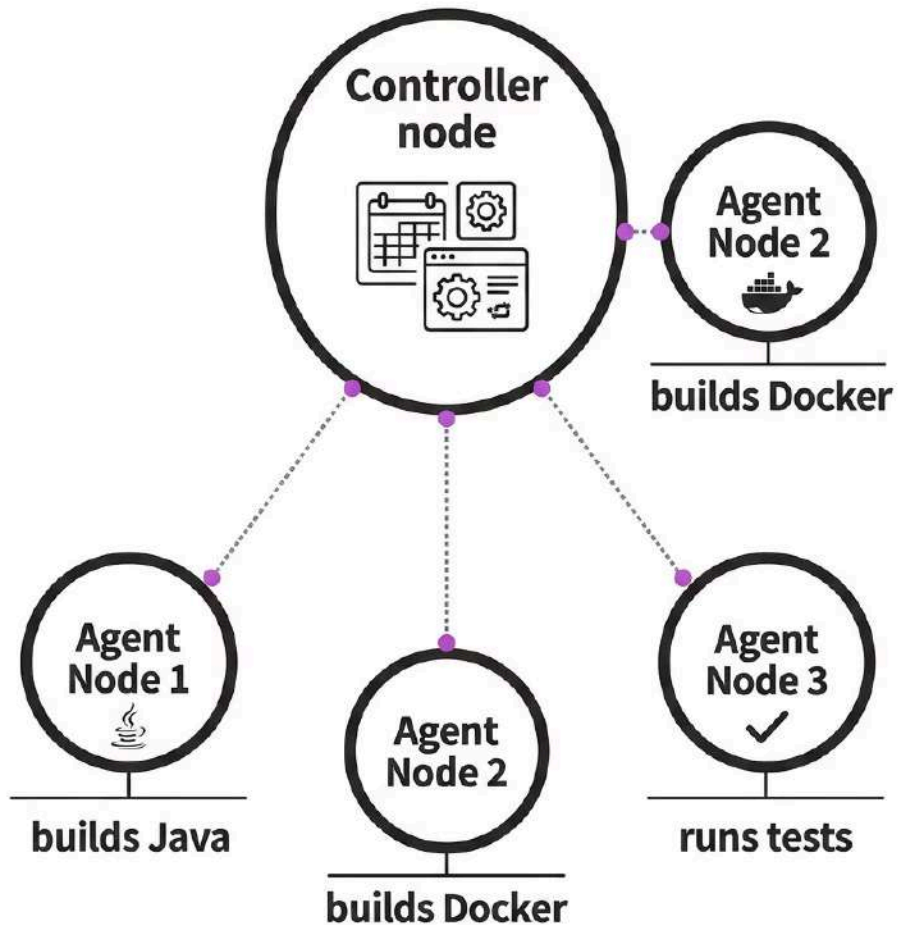
Tests run on every commit — failures are detected before they reach production



Team Consistency

Every developer's code is built and tested the same way, every time

Jenkins Architecture



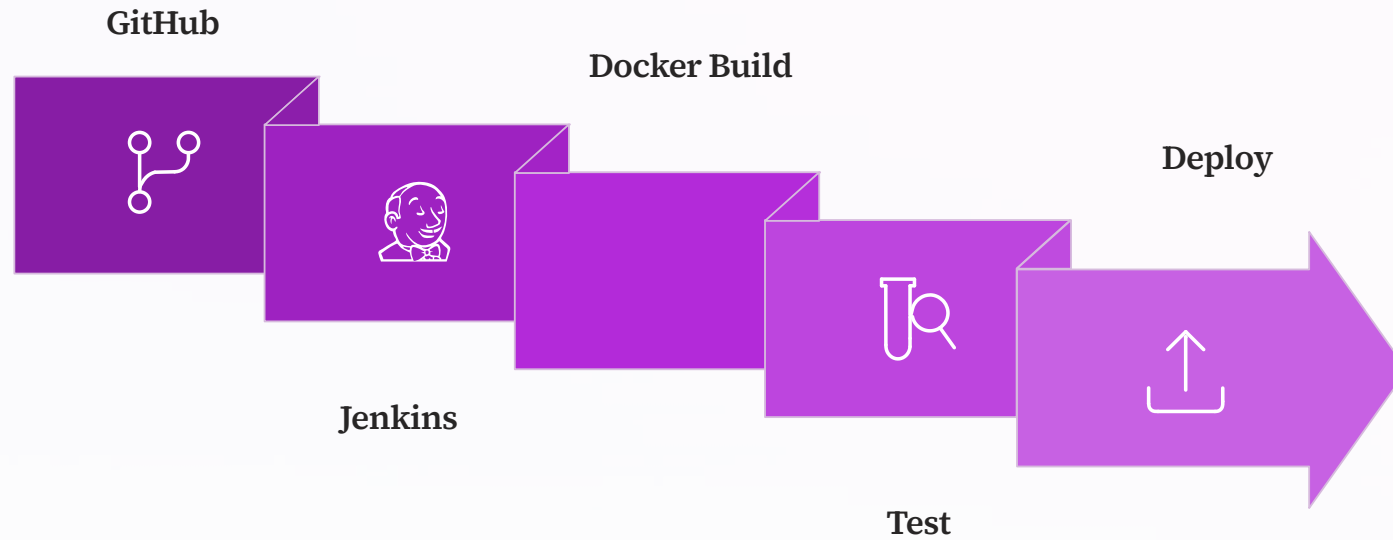
Controller + Agents

- **Controller** — Orchestrates jobs, hosts the UI, manages agents
- **Agent** — Executes the actual build/test/deploy work
- **Executor** — A slot on an agent that runs one job at a time

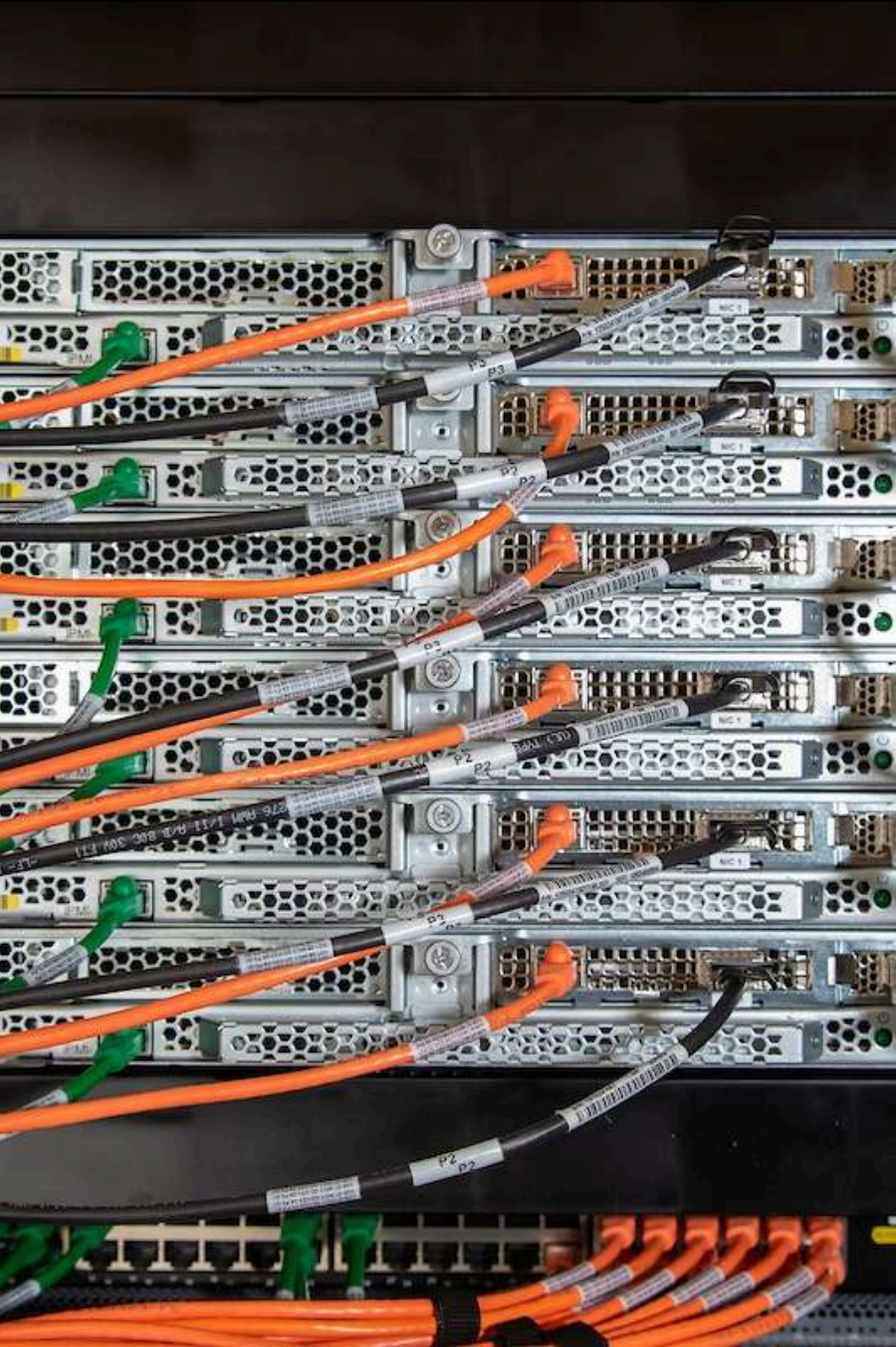


In small setups, the controller runs jobs directly. In production, always use dedicated agents to protect the controller.

The Full DevOps Pipeline



This is the pipeline you'll build in this course. By end of Day 5, every push to GitHub will automatically build, test, and deploy a Dockerized app — fully automated through Jenkins.



CHAPTER 9

Jenkins Installation

Setting up Jenkins on an EC2 instance and accessing the UI for the first time

Installing Jenkins on EC2

1

Update and install Java

`sudo apt update` then `sudo apt install openjdk-17-jre -y`
— Jenkins requires Java

2

Add Jenkins repository

Add the Jenkins GPG key and apt repository, then run `sudo apt install jenkins -y`

3

Enable and start Jenkins

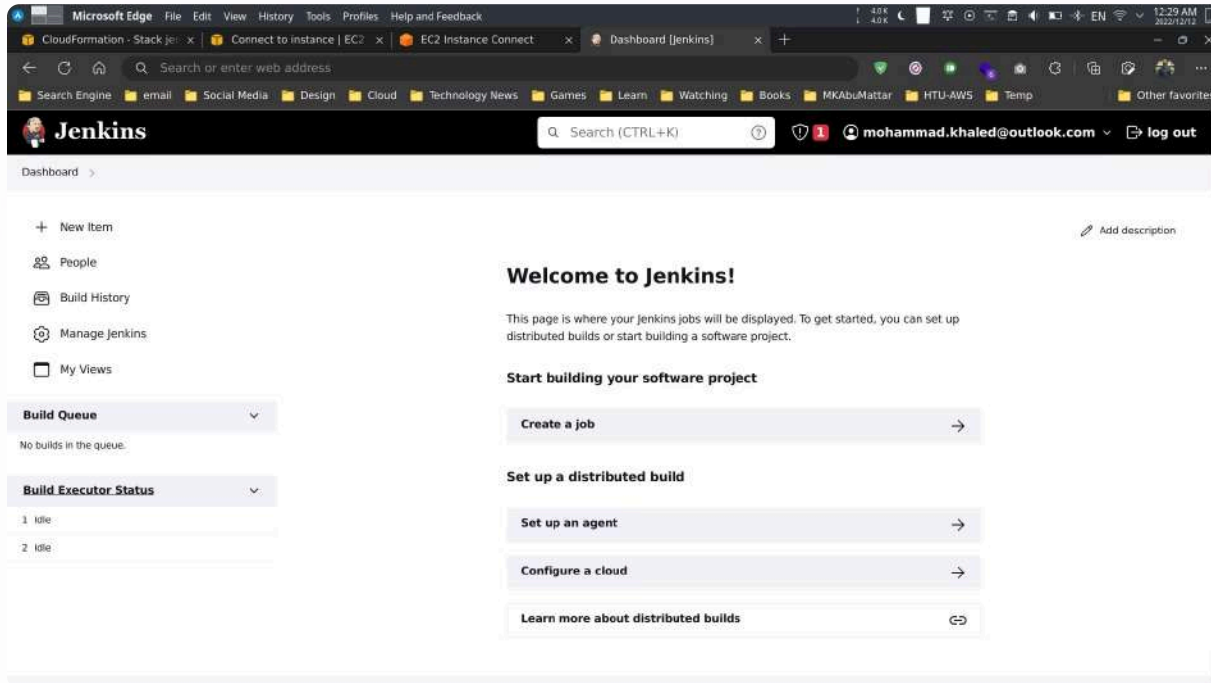
`sudo systemctl enable jenkins` and `sudo systemctl start jenkins`

4

Open port 8080 in Security Group

In AWS Console → EC2 → Security Groups → Add inbound rule:
TCP 8080

Accessing the Jenkins UI



Navigate to Jenkins in your browser

Open: `http://<EC2-PUBLIC-IP>:8080`

You'll be greeted by the **Unlock Jenkins** screen asking for an initial admin password.

Retrieve the unlock password

```
sudo cat /var/lib/jenkins/  
secrets/initialAdminPassword
```

 Copy the password and paste it into the browser to proceed to setup.

Completing Jenkins Setup

01

Unlock Jenkins

Paste the initial admin password from the terminal into the browser

03

Create Admin User

Set your username, password, and email — these are your permanent Jenkins credentials

02

Install Suggested Plugins

Click "Install suggested plugins" — Jenkins installs Git, Pipeline, and core integrations

04

Configure Instance URL

Confirm the Jenkins URL (defaults to your EC2 IP) and click Save and Finish



Expected Outcome

You should now have:

- Jenkins running on EC2 port 8080
- Admin account created and logged in
- Jenkins dashboard visible in browser
- Suggested plugins installed



Coming up on Day 4:

- Create your first Jenkins Pipeline job
- Connect Jenkins to your GitHub repository
- Trigger builds automatically on push
- Add Docker build steps to the pipeline



Day 3 complete! You've containerized an app and set up your CI/CD server.

Day 3 Key Takeaways



Docker solves environment inconsistency

Containers bundle app + dependencies for consistent runs everywhere



Images are templates; containers are instances

One image → many containers, all identical



Dockerfiles define your image

FROM → WORKDIR → COPY → RUN → CMD is the standard pattern



Jenkins automates the pipeline

Controller orchestrates agents to build, test, and deploy on every commit



Great Work Today! 🎉

You've gone from zero Docker knowledge to running containers, writing Dockerfiles, using AI assistance, and standing up Jenkins — all in one day.

SEE YOU ON DAY 4: PIPELINES & AUTOMATION